

Threaded Prime Number Search

Zhean Robby Ganituen

Task Division Schemes

By Search Range Division

The first scheme is to divide the search space itself.

If we have x threads, $(T_1, T_2, \dots, T_x)$, and we want to find all primes in $S = [0, y]$. We will first partition S into x subsequences of length $\lceil \frac{x}{y} \rceil$. And we assign a unique subsequence to each of the threads.

Since checking the divisibility of some number $i \in S$ doesn't depend on another number $j \in S$, or the subproblems are independent, the implementation is trivial.

The implementation of this scheme is in `range.go`.

By Search Range Division Implementation

```
func ByRange(cfg options.Config, printType
options.PrintConfig) []int {
    limit := cfg.Y + 1
    scope := int(math.Ceil(float64(limit) / float64(cfg.X)))

    var wg sync.WaitGroup
    var found []int
    var mu sync.Mutex

    // continued
```

This section initializes function-level variables and synchronization tools to be used:

- The search scope `int` for each chunk
- The counting semaphore `wg WaitGroup`
- The mutex lock `mu Mutex`

By Search Range Division Implementation (*cont.*)

```
for i := range cfg.X {  
    bot := i * scope  
    if bot >= limit {  
        break  
    }  
    top := min(bot+scope, limit)  
  
    wg.Add(1)  
  
    // continued
```

This section initializes thread-specific variables such as the minimum bot and maximum top number in each threads search scope.

Also in this section do we add a thread in the wg WaitGroup.

By Search Range Division Implementation (*cont.*)

```
for i := range cfg.X {  
    bot := i * scope  
    if bot >= limit {  
        break  
    }  
    top := min(bot+scope, limit)  
  
    wg.Add(1)  
  
    // continued
```

This is a for loop from $[0, \text{cfg.X}]$. Essentially, this outer loop is responsible for spawning the threads specified in the config.

By Search Range Division Implementation (*cont.*)

```
go func(workerID, minS, maxS int) {
    defer wg.Done()
    isr, err := IntStackRange(minS, maxS)
    if err != nil {
        log.Panic(err)
    }

    for !isr.IsEmpty() {
        if curr, err := isr.Pop(); err == nil {
            if TrialDivision(curr) {
                mu.Lock()
                found = append(found, curr)
                if printType == options.Now {
                    log.Printf("[Thread %d] Found prime: %d",
workerID, curr)
                }
                mu.Unlock()
            }
        }
    }
}(i, bot, top)
}
```

// continued

This section creates a Goroutine (a Go runtime managed concurrency unit) for the primality test.

By Search Range Division Implementation (*cont.*)

```
go func(workerID, minS, maxS int) {  
    defer wg.Done()  
    isr, err := IntStackRange(minS, maxS)  
    if err != nil {  
        log.Panic(err)  
    }  
  
    for !isr.IsEmpty() {  
        if curr, err := isr.Pop(); err == nil {  
            if TrialDivision(curr) {  
                mu.Lock()  
                found = append(found, curr)  
                if printType == options.Now {  
                    log.Printf("[Thread %d] Found prime: %d",  
workerID, curr)  
                }  
                mu.Unlock()  
            }  
        }  
    }  
}(i, bot, top)  
}  
  
// continued
```

The highlighted sections define the formal parameters (workerID, minS, maxS int) used by this asynchronous block.

Then, we pass the actual parameters to each parameter as i (the current index of the for loop), bot and top (which are the lower and upper bound of the search scope for this thread).

By Search Range Division Implementation (*cont.*)

```
go func(workerID, minS, maxS int) {  
    defer wg.Done()  
    isr, err := IntStackRange(minS, maxS)  
    if err != nil {  
        log.Panic(err)  
    }  
  
    for !isr.IsEmpty() {  
        if curr, err := isr.Pop(); err == nil {  
            if TrialDivision(curr) {  
                mu.Lock()  
                found = append(found, curr)  
                if printType == options.Now {  
                    log.Printf("[Thread %d] Found prime: %d",  
workerID, curr)  
                }  
                mu.Unlock()  
            }  
        }  
    }  
}(i, bot, top)  
}  
  
// continued
```

This cleans up the thread once it has terminated.

The lines below simply set up a helper data structure (this is a stack).

By Search Range Division Implementation (*cont.*)

Divisibility test proper.

```
go func(workerID, minS, maxS int) {
    defer wg.Done()
    isr, err := IntStackRange(minS, maxS)
    if err != nil {
        log.Panic(err)
    }

    for !isr.IsEmpty() {
        if curr, err := isr.Pop(); err == nil {
            if TrialDivision(curr) {
                mu.Lock()
                found = append(found, curr)
                if printType == options.Now {
                    log.Printf("[Thread %d] Found prime: %d",
workerID, curr)
                }
                mu.Unlock()
            }
        }
    }
}(i, bot, top)
}
```

// continued

By Search Range Division Implementation (cont.)

```
wg.Wait()

slices.Sort(found)
if printType == options.Later {
    log.Printf("All threads completed. Found %d primes: %v",
len(found), found)
}
return found
} // ByRange()
```